

Software Requirements Specification (SRS) Template

Project Title: AI-Powered Document Intelligence

Version: 1.0

Date:

Prepared By: Tigin Tom

Table of contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions\
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Project Scope
 - 1.5 Definitions, Acronyms, and Abbreviations
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and Dependencies
3. Specific Requirements
 - 1.1 Functional Requirements
 - 1.2 Non-Functional Requirements
4. Problem Identification and Analysis
 - 1.1 Problem Statement
 - 1.2 Objectives of the System
 - 1.3 Existing System and Its Limitations
 - 1.4 Proposed System Advantages
5. System Architecture (High-Level Design)
 - 1.1 High-Level Architecture Diagram
 - 1.2 Component Description
6. Future Enhancements / Research Directions
 - 1.1 Potential Future Enhancements
 - 1.2 Research Directions
7. References
 - 1.1 Books and Academic Sources
 - 1.2 Websites and Articles
 - 1.3 Tools and Technologies Referenced
8. AI Assistance Disclosure

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the **AI-Powered Document Intelligence Platform**, a system developed as an individual academic project for the MCA program. The goal of this system is to process and understand uploaded documents using AI techniques, enabling tasks such as document summarization, information extraction, and intelligent question answering.

This document serves to clearly state the scope, objectives, and technical expectations of the system. It is intended to “freeze” all software requirements before the design and implementation phase begins.

1.2 Document Conventions

This document follows the following conventions:

- Headings are numbered hierarchically (e.g., 1, 1.1, 1.2)
- Functional requirements are labeled as **FRxxx** (e.g., FR001)
- Non-functional requirements are labeled as **NFRxxx**
- "Shall" indicates a mandatory requirement
- "Should" indicates a recommended but optional feature
- Times are presented in 24-hour format
- Dates follow the DD-MM-YYYY format

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

- **Project Evaluators and Faculty Advisors:** To assess the clarity, feasibility, and completeness of the system’s requirements.
- **Student Developer (Myself):** To use as a guide during design and implementation.
- **External Reviewers (if applicable):** To understand the project scope and requirements.

Suggested Reading Approach:

- **Evaluators** should focus on Sections 1, 2, and 3.
- **Developers** should pay close attention to Sections 3 and 4.
- **Technical reviewers** may also review Sections 5 and 6 for system design understanding.

1.4 Project Scope

The AI-Powered Document Intelligence Platform is designed to help users interact with and understand documents through AI methods. It allows users to upload PDF documents, after which the system performs various tasks such as:

- Extracting text content
- Summarizing long documents
- Identifying key entities and concepts
- Enabling users to ask questions about the document and receive AI-generated answers

The system **will not** include:

- Web or mobile deployment
- Real-time collaboration or multi-user access
- File types other than PDFs
- Backend APIs or complex DevOps pipelines

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial Intelligence
SRS	Software Requirements Specification
NER	Named Entity Recognition
LLM	Large Language Model
PDF	Portable Document Format

RAG	Retrieval-Augmented Generation
FAISS	Facebook AI Similarity Search (used for vector search)
ChromaDB	A local vector database for storing embeddings
Streamlit	A Python-based UI framework
Mistral	An open-source language model used locally for inference
LangChain	A framework for chaining AI tools and building agent-like behavior

2. Overall Description

2.1 Product Perspective

The AI-Powered Document Intelligence Platform is a **standalone system** built for academic purposes. It does not depend on any external enterprise systems or APIs. The application runs locally on a single-user machine using Python and open-source AI libraries.

This project belongs to the **AI/Data Science stream** and focuses purely on intelligent document processing. It does not include traditional software components like multi-user support, authentication, or backend REST APIs.

2.2 Product Functions

At a high level, the system performs the following major functions:

1. **PDF Upload:** Accepts PDF documents from users through a simple interface.
2. **Text Extraction:** Reads and extracts text content from the uploaded PDFs.
3. **Document Summarization:** Provides concise summaries of lengthy documents using local AI models.
4. **Entity Extraction:** Identifies and lists key named entities (e.g., people, organizations, locations).
5. **Question Answering (Q&A):** Allows users to ask questions about the document and receive AI-generated answers using retrieval-based generation.
6. **Chat Memory Storage:** Stores user interactions and previous questions/answers for each document session using PostgreSQL.

7. **Vector-Based Search:** Stores text chunks as vector embeddings using FAISS or ChromaDB to enable semantic search during Q&A.

2.3 User Characteristics

This system is intended to be used by:

User Type	Skill Level	Description
Student Users	Basic to Intermediate	Users familiar with using simple UIs and reading documents. No technical knowledge of AI is required.
Developer (Myself)	Intermediate to Advanced	Responsible for building, maintaining, and testing the system. Familiar with Python, AI models, and data science tools.

No advanced technical knowledge is required from the end-user. The interface is built to be simple and intuitive using Streamlit.

2.4 Constraints

The system is subject to the following constraints:

- The system will run only on **local machines** (no cloud hosting).
- Only **PDF files** are supported for input (no Word, Excel, or image files).
- The AI model used (Mistral) must be **run locally via Ollama**, requiring hardware with sufficient RAM/CPU.
- **PostgreSQL** is used to store session history and will run on the local database server.
- The project is built using **Python** only (no external backend frameworks like Node.js or Java).
- No real-time collaboration or networking functionality will be implemented.

2.5 Assumptions and Dependencies

- Users will provide **legible and text-based PDFs** (not scanned images).
- The local system has Python 3.10+ and the required dependencies installed (e.g., streamlit, langchain, faiss, psycopg2, ollama).

- Ollama is assumed to be installed and running the Mistral model locally.
- FAISS or ChromaDB is installed and initialized to handle vector search.
- PostgreSQL server is properly configured for storing chat history.
- Streamlit will be used to create the user interface and run on localhost.

3. Specific Requirements

This section describes the detailed **functional** and **non-functional** requirements of the AI-Powered Document Intelligence Platform.

3.1 Functional Requirements

Below are the key system functionalities, each labeled and structured:

FR001 – PDF Upload

- **Description:** The system shall allow the user to upload PDF documents.
- **Input:** PDF file via Streamlit interface.
- **Processing:** File validation (format, size).
- **Output:** Confirmation message and stored file path.
- **Pre-conditions:** User must access the UI via a browser (localhost).
- **Post-conditions:** PDF is available for further processing.
- **Error Handling:** Display error if the file is not a PDF or is corrupted.

FR002 – Text Extraction

- **Description:** The system shall extract raw text from uploaded PDF files.
- **Input:** Uploaded PDF file.
- **Processing:** Extract text using libraries like PyMuPDF or pdfplumber.
- **Output:** Cleaned text content stored in memory or temporary file.
- **Pre-conditions:** A valid PDF must be uploaded.
- **Post-conditions:** Text is available for further AI processing.

- **Error Handling:** Display error if text cannot be extracted.

FR003 – Document Summarization

- **Description:** The system shall generate a short summary of the extracted document text.
- **Input:** Extracted text.
- **Processing:** Pass text to a locally running Mistral model via Ollama.
- **Output:** Summary displayed to the user.
- **Pre-conditions:** Document must be uploaded and text must be extracted.
- **Post-conditions:** Summary is available for review.
- **Error Handling:** Notify the user if the summarization fails.

FR004 – Named Entity Recognition (NER)

- **Description:** The system shall extract named entities (e.g., people, organizations, locations).
- **Input:** Extracted text.
- **Processing:** Use spaCy or a transformer-based NER model.
- **Output:** List of named entities categorized by type.
- **Pre-conditions:** Text must be extracted.
- **Post-conditions:** Entities are displayed in the interface.
- **Error Handling:** Return an empty list or error message if no entities are found.

FR005 – Question Answering (Q&A) with Retrieval

- **Description:** The system shall allow the user to ask questions related to the uploaded document and return an accurate answer.
- **Input:** User query (text).
- **Processing:**
 - Retrieve relevant chunks using FAISS or ChromaDB

- Pass context and query to Mistral for generation
- **Output:** AI-generated answer.
- **Pre-conditions:** A document must be uploaded and embedded.
- **Post-conditions:** Response shown on screen.
- **Error Handling:** Notify the user if no relevant answer is found.

FR006 – Chat History Storage

- **Description:** The system shall store each question and answer for the current session.
- **Input:** User question and generated answer.
- **Processing:** Insert chat log into PostgreSQL database.
- **Output:** Persisted chat history retrievable per document session.
- **Pre-conditions:** Database must be connected and configured.
- **Post-conditions:** Chat history can be reviewed later.
- **Error Handling:** Notify user if chat logging fails.

3.2 Non-Functional Requirements

NFR001 – Performance Requirements

- The system shall load and display document summaries within **5 seconds** for documents under 10 pages.
- The response time for Q&A shall not exceed **7 seconds** under normal conditions (Mistral inference).
- Vector search operations shall complete in **under 1 second**.

NFR002 – Security Requirements

- No user authentication is required, but the system shall not allow execution of arbitrary code via user inputs.
- Uploaded documents and chat data shall remain on the local machine and never be sent to external APIs.

NFR003 – Usability Requirements

- The user interface shall be designed for **non-technical users** with clear instructions and minimal input requirements.
- The interface shall display helpful messages for errors or missing inputs.
- Chat history shall be viewable during the session in a clean layout.

4. Problem Identification and Analysis

This section outlines the real-world problem addressed by the system, the motivation behind the project, and the challenges users face without such a solution.

4.2 Objectives of the System

The primary goal of the system is to build an **AI-powered document intelligence platform** that allows users to:

- Upload PDF documents
- Automatically summarize long text
- Extract key entities
- Ask questions and receive intelligent, context-aware answers
- Retain chat history across sessions

This aims to reduce the user's time and effort in understanding documents and improve accessibility to dense or technical content.

4.3 Existing System and Its Limitations

Most existing PDF readers (like Adobe Acrobat or Chrome PDF Viewer):

- Only allow **viewing and basic search**
- Do not provide any **text summarization** or **Q&A capabilities**
- Require the user to read and understand everything manually

Even some commercial AI-based tools are:

- **Expensive** (e.g., ChatGPT Plus)

- **Require internet connectivity**
- Do not allow **local data privacy**

4.4 Proposed System Advantages

The proposed system addresses these issues by:

- Using **free and open-source tools**
- Running **fully offline**, respecting privacy
- Providing **AI assistance** for summarization, entity extraction, and Q&A
- Using a **simple Streamlit interface** that works on any local machine

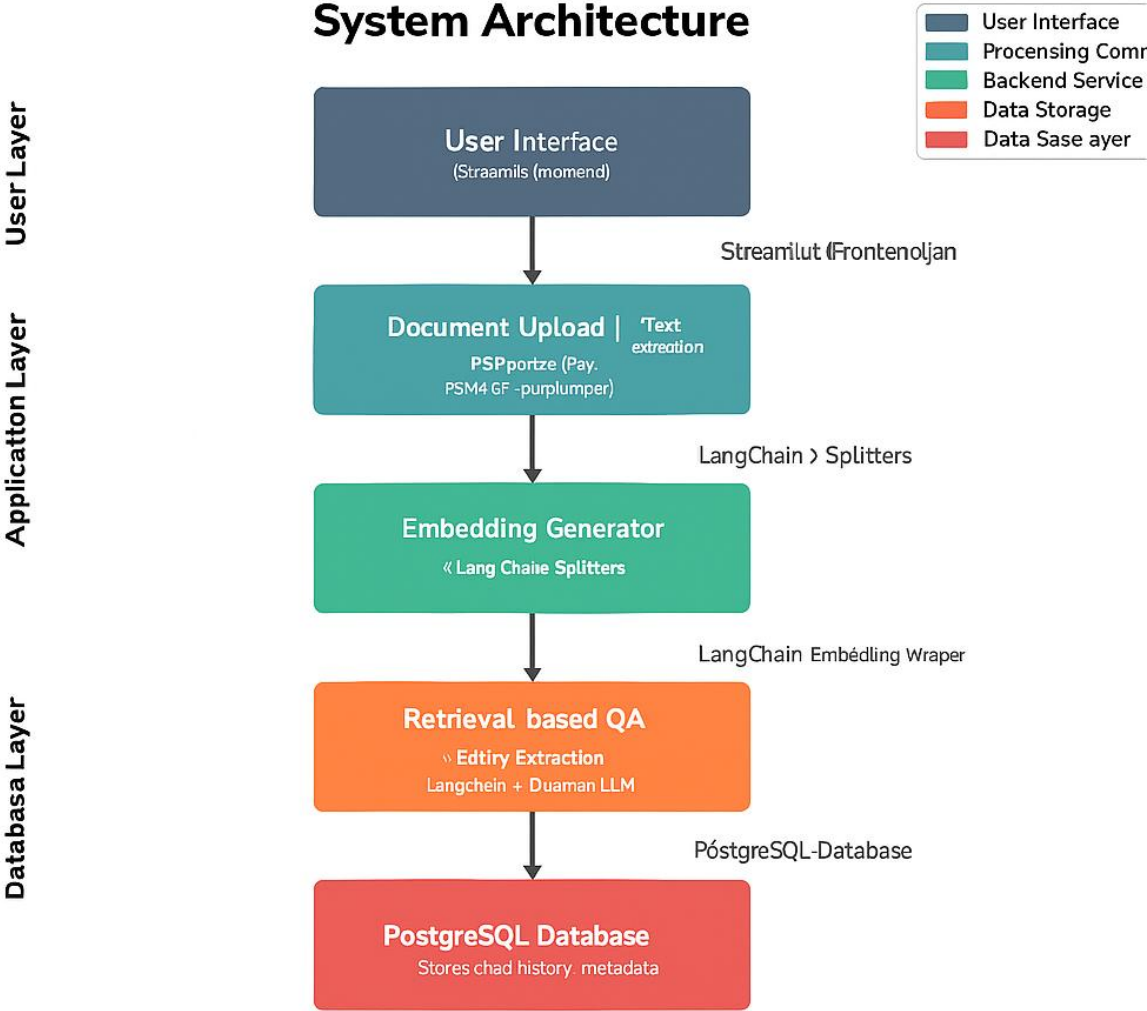
5. System Architecture (High-Level Design)

This section provides a simplified view of how the system is structured and how its major components interact.

5.1 High-Level Architecture Diagram

Here's a textual representation of the architecture (you can convert this into a block diagram for your final report):

System Architecture



5.2 Component Description

Component	Description
Streamlit UI	Allows users to upload documents, interact with summaries, ask questions, and view answers.
PDF Parser	Extracts raw text from uploaded documents.
Text Splitter	Breaks large documents into manageable chunks for embedding and processing.

Embeddings Generator	Uses Mistral (via Ollama) to convert text chunks into vector representations.
FAISS Vector Store	Stores embeddings for similarity-based document retrieval.
LLM (Mistral)	Performs Q&A, summarization, and entity recognition based on user queries.
PostgreSQL	Maintains session history and past interactions.

6. Future Enhancements / Research Directions

Although the initial implementation of the AI-Powered Document Intelligence Platform focuses on core features like summarization, Q&A, and entity extraction, several future enhancements and research directions can significantly improve its functionality and impact.

6.1 Potential Future Enhancements

Feature	Description
Multilingual Support	Extend the platform to support documents in multiple languages using language-agnostic models.
Named Entity Linking	Enhance entity extraction by linking entities to external knowledge bases (e.g., Wikidata).
Document Classification	Integrate automatic categorization of uploaded documents (e.g., invoice, contract, report).
User Authentication & Roles	Add secure login with access roles for different types of users (e.g., admin, reviewer, guest).
Custom Model Training	Allow fine-tuning of models on domain-specific documents to improve accuracy.
OCR Integration	Enable the system to process scanned documents or images using Optical Character Recognition.

Analytics Dashboard	Provide users with a dashboard for tracking document statistics and system usage.
Summarization Tuning Options	Let users choose summary length, style (bullet points vs. paragraph), or focus (e.g., financial summary, legal terms).

6.2 Research Directions

Research Area	Description
Document Reasoning and Inference	Explore the ability of LLMs to infer hidden facts or relationships from long-form documents.
Efficient Vector Search at Scale	Investigate scalable indexing and retrieval strategies for large document corpora.
Context Preservation	Address challenges in preserving long-context across multiple document sections during summarization or Q&A.
Evaluation Metrics for QA Accuracy	Define rigorous benchmarks to evaluate correctness and completeness of model-generated answers.
Human-in-the-Loop Feedback Loops	Incorporate user corrections to continuously improve the system's performance over time.

7. References

This section lists the sources, tools, frameworks, and literature referred to during the planning and requirements gathering of the AI-Powered Document Intelligence Platform.

7.1 Books and Academic Sources

1. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing* (3rd ed.) – Draft chapters. Stanford University.
2. Aggarwal, C. C. (2018). *Neural Networks and Deep Learning: A Textbook*. Springer.

3. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

7.2 Websites and Articles

1. LangChain Documentation: <https://docs.langchain.com/>
2. Streamlit Documentation: <https://docs.streamlit.io/>
3. FAISS – Facebook AI Similarity Search: <https://github.com/facebookresearch/faiss>
4. ChromaDB Docs: <https://docs.trychroma.com/>
5. PostgreSQL Official Docs: <https://www.postgresql.org/docs/>
6. Mistral Model – Ollama: <https://ollama.com/library/mistral>
7. HuggingFace Blog and Model Hub: <https://huggingface.co/blog>
8. spaCy for NLP: <https://spacy.io/>

7.3 Tools and Technologies Referenced

- **Python 3.11+** – Primary programming language.
- **Streamlit** – Frontend development framework for rapid prototyping.
- **LangChain** – LLM orchestration for chaining and retrieval-based QA.
- **FAISS / ChromaDB** – Vector database for semantic search.
- **PostgreSQL** – Used for structured memory and chat history.
- **Ollama** – For running local LLMs like Mistral.
- **scikit-learn, Pandas, NumPy** – Used for processing and basic ML/NLP pipelines.

8. AI Assistance Disclosure

This Software Requirements Specification (SRS) document was developed individually by the student **Tigin Tom** as part of the MCA academic project. AI tools were used **only in a limited capacity** to assist with:

- Brainstorming the initial structure of the SRS.
- Improving clarity in some non-technical sections (e.g., usability, performance attributes).
- Proofreading grammar and consistency after drafting each section.

The **core requirements, analysis, system architecture, and all major content** in this SRS are **original work by the author** and reflect personal understanding, problem-solving, and domain knowledge.

Tools used: **ChatGPT-4 (for outline assistance), Grammarly Free (for grammar checks)**
All references and technical frameworks mentioned were manually researched and verified.